
분류/회귀 데이터 셋 지도학습

팀	DF_3조
---	-------

분류 데이터셋

- 데이터셋 명

kaggle - Diabetes Health Indicators Dataset
- 데이터 배경

당뇨병은 대표적인 만성 질환으로, 조기 발견과 생활 습관 개선을 통해 예방 및 관리가 가능하다.

본 데이터는 미국 CDC의 BRFSS 2015 설문조사를 기반으로 한 헬스케어 데이터로, 개인의 건강 상태와 생활 습관 정보를 포함한다.
- 프로젝트 목표

당뇨 위험군 예측하는 분류 모델 구축 : Class 0 (정상) / Class1 (당뇨 위험군)
- 데이터셋 명

diabetes_binary_health_indicators_BRFSS2015.csv

diabetes_binary_5050split_health_indicators_BRFSS2015.csv	diabetes_binary_health_indicators_BRFSS2015.csv	diabetes_012_health_indicators_BRFSS2015.csv
2clss 분류	2class 분류	3class 분류
7만개 데이터	25만개의 데이터	25만개 데이터
클래스 불균형 X	클래스 불균형 0	클래스 불균형 0

데이터 특징

25,3680 rows x 22 columns

```
<class 'pandas.DataFrame'>
RangeIndex: 253680 entries, 0 to 253679
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Diabetes_binary                       253680 non-null float64
1   HighBP                               253680 non-null float64
2   HighChol                             253680 non-null float64
3   CholCheck                            253680 non-null float64
4   BMI                                   253680 non-null float64
5   Smoker                               253680 non-null float64
6   Stroke                               253680 non-null float64
7   HeartDiseaseorAttack                 253680 non-null float64
8   PhysActivity                         253680 non-null float64
9   Fruits                               253680 non-null float64
10  Veggies                              253680 non-null float64
11  HvyAlcoholConsump                   253680 non-null float64
12  AnyHealthcare                       253680 non-null float64
13  NoDocbcCost                         253680 non-null float64
14  GenHlth                             253680 non-null float64
15  MentHlth                            253680 non-null float64
16  PhysHlth                            253680 non-null float64
17  DiffWalk                             253680 non-null float64
18  Sex                                  253680 non-null float64
19  Age                                  253680 non-null float64
20  Education                            253680 non-null float64
21  Income                              253680 non-null float64
dtypes: float64(22)
memory usage: 42.6 MB
```

Diabetes_binary 타겟 변수 Y 설정

당뇨 위험군을 나타내는 변수

0 : 정상

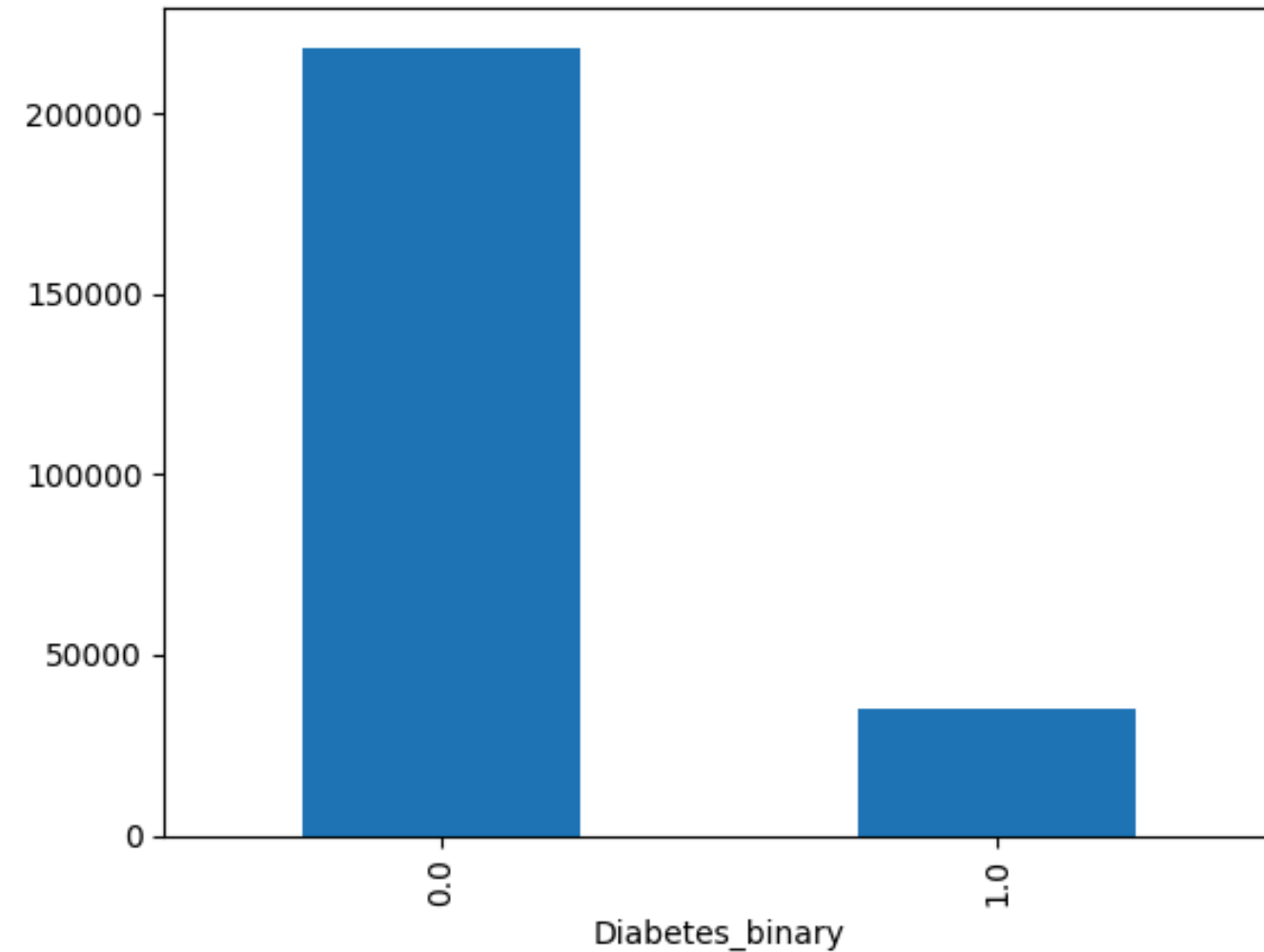
1 : 당뇨 위험군

Null 결측치 없음

```
df.isnull()
```

[illegible]

데이터 불균형



	0 (정상)	1 (당뇨 위험군)
데이터 개수	218334	35346
데이터 확률	86%	14%

데이터에서 정상이 86%로 대부분을 차지하기 때문에, 단순 0만 예측해도 높은 정확도가 나올 가능성이 있고, 1(당뇨 위험군) 데이터를 제대로 학습하지 못할 확률이 큼

=> 소수 클래스에 가중치를 부여하거나, 평가 지표를 Accuracy 대신 Recall(재현율), F1-score 활용하여 불균형 문제 해결

회귀 데이터셋

데이터셋 명	kaggle - Starbucks Customer Ordering Patterns
데이터 배경	주문 채널(모바일 앱, 드라이브 스루, 매장, 키오스크), 고객 연령대, 주문시간, 품목 수 및 맞춤설정 등을 반영하여 스타벅스 고객의 주문 패턴을 분석하기 위해 생성된 시뮬레이션 데이터
데이터 특징	총 100,000건의 거래 데이터를 포함하며,, 2024 ~ 2025년(2년)의 기간을 반영. 디지털 벤티 효과(Digital Venti Effect)를 포함하여 모바일 앱 사용자가 기존 매장 구매자에 비해 평균 주문 금액, 맞춤 설정 빈도가 더 높으며, 연령별 이용 양상이 뚜렷.
프로젝트 목표	1. 시간대, 매장 위치 유형, 커스텀 횟수 등의 데이터를 통해 고객의 총 지출액(total_spend)을 예측하는 회귀 모델 구축. 2. 총 지출액(매출)을 가장 잘 예측하는 핵심 요인 도출 및 '디지털 벤티 효과' 등 고객 주문 패턴에 대한 비즈니스 인사이트 확보.
데이터셋 명	starbucks_customer_ordering_patterns.csv

데이터 특징

100,000 rows × 20 columns

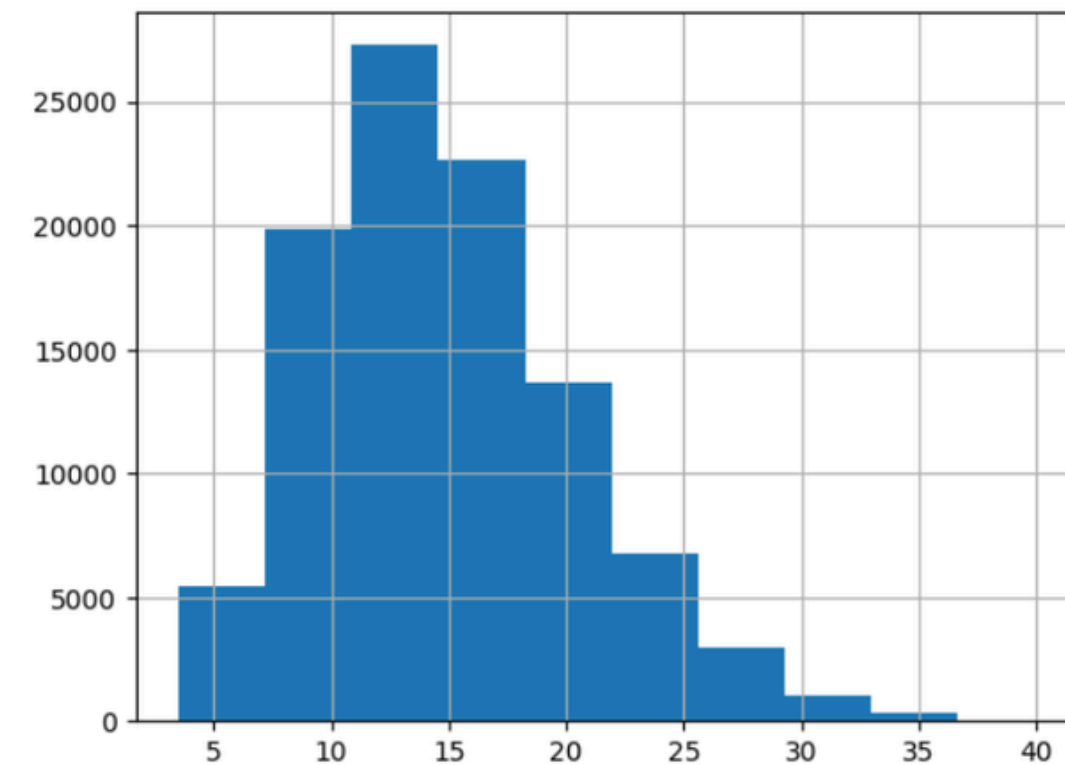
```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           100000 non-null  str
1   order_id              100000 non-null  str
2   order_date            100000 non-null  str
3   order_time            100000 non-null  str
4   day_of_week           100000 non-null  str
5   order_channel         100000 non-null  str
6   store_id              100000 non-null  str
7   store_location_type   100000 non-null  str
8   region                100000 non-null  str
9   customer_age_group    100000 non-null  str
10  customer_gender        100000 non-null  str
11  is_rewards_member      100000 non-null  bool
12  cart_size              100000 non-null  int64
13  num_customizations     100000 non-null  int64
14  total_spend            100000 non-null  float64
15  fulfillment_time_min   100000 non-null  float64
16  drink_category         100000 non-null  str
17  has_food_item          100000 non-null  bool
18  order_ahead            100000 non-null  bool
19  customer_satisfaction  100000 non-null  int64
dtypes: bool(3), float64(2), int64(3), str(12)
memory usage: 13.3 MB
```

total_spend 타겟 변수 Y 설정

총 지출액 예측

Null 결측치 없음

이상치 확인



count	100000.000000
mean	14.866771
std	5.506800
min	3.510000
25%	10.837500
50%	14.170000
75%	18.180000
max	40.310000

평균 값들 보다 튀는 값(35~40) 이상치가 전체 모델을 왜곡할 가능성 있음

=> 추후 제거 필요

분류

- 01 데이터 프레임 불러오기
- 02 훈련 데이터, 테스트 데이터 생성
- 03 데이터 스케일링
- 04 베이스라인 모델 학습 및 기본 성능
- 05 교차 검증
- 06 비교 결론

분류 데이터 프레임 불러오기

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler

pd.read_csv('diabetes_processed.csv') #데이터 프레임 불러오기
```

HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	Veggies	HvyAlcoholConsump	NoDocbcCost	GenHlth	MentHlth	PhysHlth	DiffWalk	Sex	Age	Education	Income	Risk_Sum
1.0	1.0	1.0	40.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	5.0	18.0	15.0	1.0	0.0	9.0	4.0	3.0	2.0
0.0	0.0	0.0	25.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	3.0	0.0	0.0	0.0	0.0	7.0	6.0	1.0	0.0
1.0	1.0	1.0	28.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	5.0	30.0	30.0	1.0	0.0	9.0	4.0	8.0	2.0
1.0	0.0	1.0	27.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	2.0	0.0	0.0	0.0	0.0	11.0	3.0	6.0	1.0
1.0	1.0	1.0	24.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	2.0	3.0	0.0	0.0	0.0	11.0	5.0	4.0	2.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1.0	1.0	1.0	45.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	3.0	0.0	5.0	0.0	1.0	5.0	6.0	7.0	2.0
1.0	1.0	1.0	18.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	4.0	0.0	0.0	1.0	0.0	11.0	2.0	4.0	2.0
0.0	0.0	1.0	28.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	2.0	5.0	2.0	0.0
1.0	0.0	1.0	23.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	3.0	0.0	0.0	0.0	1.0	7.0	5.0	1.0	1.0
1.0	1.0	1.0	25.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0	0.0	9.0	6.0	2.0	3.0

훈련 데이터, 테스트 데이터 생성

```
# 테스트 데이터 20%로 설정, X, Y를 토대로 훈련 데이터, 테스트 데이터 생성
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
print(X_train, X_test)
print(Y_train, Y_test)
```

[202944 rows x 21 columns]											[50736 rows x 21 columns]										
HeartDiseaseorAttack PhysActivity Fruits Veggies #											HighBP HighChol CholCheck BMI Smoker Stroke #										
31141	0.0	1.0	1.0	1.0	219620	0.0	0.0	1.0	21.0	0.0	0.0										
98230	0.0	1.0	0.0	1.0	132821	1.0	1.0	1.0	28.0	0.0	0.0										
89662	0.0	1.0	1.0	1.0	151862	0.0	0.0	1.0	24.0	0.0	0.0										
208255	0.0	1.0	1.0	1.0	139717	0.0	0.0	1.0	27.0	1.0	0.0										
233415	0.0	1.0	1.0	1.0	239235	0.0	1.0	1.0	31.0	1.0	0.0										
...	...	...	...	...	...	...	...	...	...	...	...										
119879	0.0	1.0	1.0	0.0	169513	1.0	0.0	1.0	29.0	1.0	0.0										
103694	0.0	1.0	0.0	1.0	182415	0.0	0.0	1.0	25.0	0.0	0.0										
131932	0.0	1.0	1.0	1.0	109739	0.0	1.0	1.0	28.0	0.0	0.0										
146867	0.0	0.0	1.0	1.0	181671	0.0	0.0	1.0	24.0	1.0	0.0										
121958	0.0	0.0	1.0	1.0	202118	0.0	0.0	1.0	23.0	0.0	0.0										
HeartDiseaseorAttack PhysActivity Fruits Veggies #											HvyAlcoholConsump NoDocbcCost GenHlth MentHlth PhysHlth DiffWalk #										
31141	0.0	0.0	2.0	0.0	0.0	219620	0.0	0.0	1.0	1.0											
98230	0.0	0.0	3.0	0.0	0.0	132821	0.0	1.0	1.0	1.0											
89662	0.0	0.0	2.0	0.0	5.0	151862	0.0	1.0	1.0	1.0											
208255	0.0	0.0	1.0	0.0	0.0	139717	0.0	1.0	0.0	1.0											
233415	0.0	0.0	3.0	0.0	0.0	239235	0.0	0.0	1.0	1.0											
...	...	...	...	...	...	...	...	...	...	...											
119879	0.0	1.0	1.0	15.0	0.0	0.0	169513	0.0	1.0	1.0	1.0										
103694	0.0	0.0	3.0	0.0	0.0	0.0	182415	0.0	1.0	1.0	1.0										
131932	0.0	0.0	2.0	0.0	3.0	0.0	109739	0.0	1.0	1.0	1.0										
146867	0.0	1.0	2.0	0.0	0.0	0.0	181671	0.0	0.0	0.0	1.0										
121958	0.0	0.0	3.0	15.0	0.0	1.0	202118	0.0	1.0	1.0	1.0										
Sex Age Education Income Risk_Sum											HvyAlcoholConsump NoDocbcCost GenHlth MentHlth PhysHlth DiffWalk #										
31141	1.0	12.0	6.0	8.0	1.0	219620	0.0	0.0	3.0	3.0	7.0	0.0									
98230	1.0	8.0	5.0	8.0	0.0	132821	0.0	0.0	3.0	0.0	0.0	0.0									
89662	1.0	12.0	5.0	6.0	2.0	151862	0.0	0.0	1.0	0.0	0.0	0.0									
208255	1.0	5.0	6.0	7.0	1.0	139717	1.0	0.0	2.0	3.0	0.0	0.0									
233415	0.0	12.0	4.0	6.0	1.0	239235	0.0	1.0	4.0	27.0	27.0	1.0									
...	...	...	...	...	...	...	...	...	...	...	...	...									
119879	0.0	5.0	4.0	1.0	1.0	169513	0.0	0.0	3.0	0.0	10.0	0.0									
103694	1.0	11.0	6.0	7.0	2.0	182415	0.0	0.0	2.0	1.0	10.0	0.0									
131932	0.0	9.0	6.0	8.0	1.0	109739	0.0	0.0	3.0	3.0	0.0	0.0									
146867	0.0	5.0	6.0	6.0	0.0	181671	0.0	1.0	4.0	0.0	0.0	0.0									
121958	0.0	11.0	5.0	6.0	1.0	202118	0.0	0.0	2.0	0.0	0.0	0.0									
Sex Age Education Income Risk_Sum											Name: Diabetes_binary, Length: 202944, dtype: float64										
31141	0.0	12.0	6.0	8.0	2.0	219620	0.0	7.0	4.0	2.0	0.0	0.0									
98230	0.0	8.0	5.0	8.0	0.0	132821	0.0	12.0	6.0	6.0	2.0	0.0									
89662	0.0	12.0	5.0	6.0	2.0	151862	0.0	12.0	6.0	6.0	2.0	0.0									
208255	0.0	5.0	6.0	7.0	1.0	139717	0.0	12.0	6.0	6.0	2.0	0.0									
233415	0.0	12.0	4.0	6.0	1.0	239235	0.0	12.0	6.0	6.0	2.0	0.0									
...	...	...	...	...	...	...	...	...	...	...	...	...									
119879	0.0	5.0	4.0	1.0	1.0	169513	0.0	5.0	4.0	1.0	1.0	0.0									
103694	1.0	11.0	6.0	7.0	2.0	182415	0.0	11.0	6.0	7.0	2.0	0.0									
131932	0.0	9.0	6.0	8.0	1.0	109739	0.0	9.0	6.0	8.0	1.0	0.0									
146867	0.0	5.0	6.0	6.0	0.0	181671	0.0	5.0	6.0	6.0	0.0	0.0									
121958	0.0	11.0	5.0	6.0	1.0	202118	0.0	11.0	5.0	6.0	1.0	0.0									
Name: Diabetes_binary, Length: 50736, dtype: float64																					

데이터 스케일링

```
# 변수 간 값의 범위 차이를 줄이기 위해 표준화 사용  
# 값의 범위가 큰 세 변수 표준화
```

```
scale_cols = ['BMI', 'MentHlth', 'PhysHlth']
```

```
scaler = StandardScaler()  
X_train[scale_cols] = scaler.fit_transform(X_train[scale_cols])  
X_test[scale_cols] = scaler.transform(X_test[scale_cols])
```

BMI	BMI
50736.000000	50736.000000
28.424354	0.007938
6.593808	0.997193
12.000000	-2.475946
24.000000	-0.661165
27.000000	-0.207470
31.000000	0.397457
98.000000	10.529983

GenHlth	MentHlth
50736.000000	50736.000000
2.509638	3.152988
1.069436	7.357394
1.000000	0.000000
2.000000	0.000000
2.000000	0.000000
3.000000	2.000000
5.000000	30.000000

MentHlth	PhysHlth
50736.000000	50736.000000
-0.005350	-0.003846
0.990678	0.998309
-0.429902	-0.487197
-0.429902	-0.487197
-0.429902	-0.487197
-0.160600	-0.143195
3.609619	2.952820

베이스라인 모델 학습 및 기본 성능

```
#fit(), predict() 사용하여 예측 수행
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, Y_train)

Y_pred = rf.predict(X_test)
base_accuracy = accuracy_score(Y_test, Y_pred)
print(f'기본 정확도: {base_accuracy:.4f}')
```

기본 정확도: 0.8599

RandomForest

- 앙상블 기법인 RandomForestClassifier를 활용하여 당뇨 발병 여부 예측 모델 구축
- Train/Test 데이터를 분리하여 초기 학습(fit) 및 예측(predict) 수행
- 초기 예측 결과: 베이스라인 모델 기준 정확도(Accuracy) 0.8599 (약 86%) 달성

K-Fold & StratifiedKFold 교차 검증

```
# KFold
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
kfold_scores = cross_val_score(rf, X_train, Y_train, cv=kfold, scoring='accuracy')
print(f'KFold 각 폴드 정확도: {kfold_scores}')
print(f'KFold 평균 정확도: {kfold_scores.mean():.4f}')
```

KFold 각 폴드 정확도: [0.86121856 0.85799108 0.85616793 0.85944468 0.8593673]
KFold 평균 정확도: 0.8588

```
# StratifiedKFold
# 당뇨 데이터는 클래스 불균형이 있기 때문에
# 각 폴드에서 클래스 비율을 유지하는 StratifiedKFold가 더 적합
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
skfold_scores = cross_val_score(rf, X_train, Y_train, cv=skfold, scoring='accuracy')
print(f'\nStratifiedKFold 각 폴드 정확도: {skfold_scores}')
print(f'StratifiedKFold 평균 정확도: {skfold_scores.mean():.4f}')
```

StratifiedKFold 각 폴드 정확도: [0.85843455 0.85974032 0.85865629 0.85897657 0.85791367]
StratifiedKFold 평균 정확도: 0.8587

교차 검증을 통한 모델 안정성 확보

- 특정 데이터 셋에만 과적합(Overfitting)되는 것을 방지하기 위해 5개의 폴드(K=5)로 나누어 교차 검증 수행
- 데이터 특성 반영: 당뇨 데이터의 경우 정상과 당뇨 환자 비율이 다른 '클래스 불균형(Class Imbalance)'이 존재함.
- 따라서 단순 무작위 분할(KFold) 대신, 각 폴드마다 타겟 변수의 비율을 원본과 동일하게 유지하는 계층적 교차 검증 (StratifiedKFold)을 주력으로 적용하여 신뢰도 향상.

비교 결론

[1. 교차 검증 결과 수치 비교]

검증 방식 (Validation Method)	평균 정확도 (Accuracy)
기본 베이스라인 (Baseline)	0.8599
KFold 교차 검증	0.8588
StratifiedKFold 교차 검증	0.8587

[2. 최종 성능 채택 기준]

- 과적합 배제 → 교차 검증 점수 채택
- StratifiedKFold가 KFold보다 폴드 간 편차 작음 (0.001 vs 0.005)
- 클래스 불균형 데이터 → StratifiedKFold 최종 채택

회귀

- 01 회귀 데이터 프레임 불러오기
- 02 훈련 데이터, 테스트 데이터 생성
- 03 데이터 스케일링
- 04 데이터 표준화&변수 분포 시각화
- 05 베이스라인 모델 학습 및 기본 성능
- 06 KFold 교차검증 결과 및 분석
- 07 grid_search 하이퍼파라미터 설정

회귀 데이터 프레임 불러오기

```
df_loaded = pd.read_csv("starbucks_preprocessed.csv") #데이터 프레임 불러오기
```

order_time	is_rewards_member	cart_size	num_customizations	total_spend	fulfillment_time_min	has_food_item	customer_satisfaction	day_of_week_Sat	day_of_week_Tue
08:47	False	5	0	14.48	8.2	False	4	False	False
08:02	True	1	3	9.52	5.4	False	4	False	False
05:40	False	2	1	9.32	4.9	False	5	False	False
15:10	True	2	1	9.55	3.5	False	4	False	True
07:38	False	3	1	12.24	4.1	False	3	False	True
...	...	...	...	...	...	...	...	...	...
17:08	True	4	0	15.18	3.2	False	4	False	False
08:07	True	5	2	16.71	5.5	False	3	True	False
16:05	False	3	0	9.12	3.3	False	1	True	False
18:32	True	5	1	18.02	1.9	False	2	False	False
13:18	False	1	0	5.29	3.1	True	5	False	False

훈련 데이터, 테스트 데이터 생성

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler

# 테스트 데이터 20%로 설정, X, Y를 토대로 훈련 데이터, 테스트 데이터 생성

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
print(X_train, X_test)
print(Y_train, Y_test)
```


데이터 스케일링

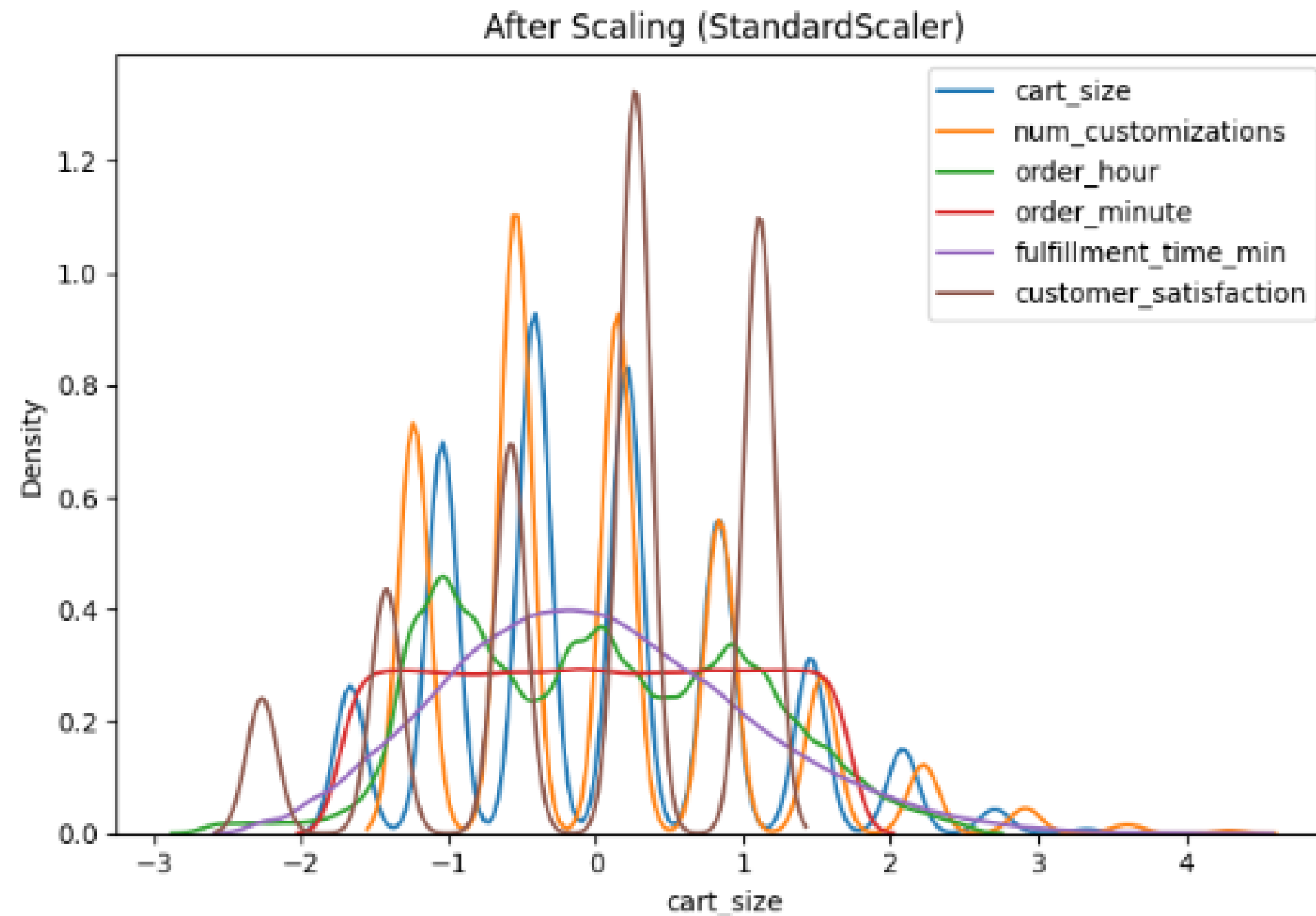
```
# StandardScaler 표준화를 이용하여 변수 간 값의 범위 차이를 줄이기

scale_cols = ['cart_size', 'num_customizations', 'order_hour', 'order_minute', 'fulfillment_time_min', 'customer_satisfaction', ]

scaler = StandardScaler() # 평균0, 표준편차 1인 값으로 표준화 실행
X_train[scale_cols] = scaler.fit_transform(X_train[scale_cols])
X_test[scale_cols] = scaler.transform(X_test[scale_cols]) # test는 fit 없음
```

cart_size	num_customizations	fulfillment_time_min	customer_satisfaction	order_hour	order_minute
19717.000000	19717.000000	19717.000000	19717.000000	19717.000000	19717.000000
0.013232	-0.000736	0.004524	-0.003766	-0.012796	-0.019782
1.014857	1.003501	0.991808	0.997817	0.996393	0.993865
-1.666737	-1.236796	-2.277066	-2.266070	-2.560461	-1.707370
-1.042205	-0.545311	-0.735284	-0.578659	-0.821160	-0.898707
0.206860	0.146174	-0.028634	0.265046	0.048490	-0.032282
0.831393	0.837658	0.613775	1.108751	0.918140	0.834143
3.954056	4.295082	4.018543	1.108751	2.440028	1.700567

데이터 (스케일링)표준화 후 변수 분포 시각화



- 평균 0, 표준편차 1로 데이터 표준화 진행

=> 어떤 변수가 total_spend에 영향을 크게 주는지 확인

=> 모든 변수 범위가 -3 ~ 4로 통일됨

베이스라인 모델 학습 및 기본 성능

```
▶ lr = LinearRegression()
  lr.fit(X_train, Y_train)

  Y_pred = lr.predict(X_test)

  mse = mean_squared_error(Y_test, Y_pred)
  r2 = r2_score(Y_test, Y_pred)

  print(f'기본 MSE: {mse:.4f}')
  print(f'기본 R² : {r2:.4f}')
```

```
... 기본 MSE: 0.9351
  기본 R² : 0.9652
```

LinearRegression 예측

- LinearRegression을 활용하여 스타벅스 주문 데이터 기반 예측 모델 구축
- Train/Test 데이터를 분리하여 초기 학습(fit) 및 예측(predict) 수행
- 초기 예측 결과: MSE = 0.9351, $R^2 = 0.9652$ (데이터 변동의 약 96.5% 설명)

KFold 교차검증 결과 및 분석

```
▶ kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# 회귀는 R² 로 비교 (1에 가까울수록 좋음)
kfold_r2 = cross_val_score(lr, X_train, Y_train, cv=kfold, scoring='r2')
# MSE는 음수로 반환되므로 부호 반전
kfold_mse = -cross_val_score(lr, X_train, Y_train, cv=kfold, scoring='neg_mean_squared_error')

print(f'KFold R² 각 폴드: {kfold_r2}')
print(f'KFold R² 평균: {kfold_r2.mean():.4f}')
print(f'KFold MSE 각 폴드: {kfold_mse}')
print(f'KFold MSE 평균: {kfold_mse.mean():.4f}')

print('\n===== 정확도 비교 =====')
print(f'기본 R² : {r2:.4f}')
print(f'KFold R² 평균: {kfold_r2.mean():.4f}')
print(f'기본 MSE: {mse:.4f}')
print(f'KFold MSE 평균: {kfold_mse.mean():.4f}')

... KFold R² 각 폴드: [0.96432699 0.96511886 0.96469856 0.964591 0.96444082]
KFold R² 평균: 0.9646
KFold MSE 각 폴드: [0.93823762 0.92752522 0.93637892 0.94248008 0.93331899]
KFold MSE 평균: 0.9356

===== 정확도 비교 =====
기본 R² : 0.9652
KFold R² 평균: 0.9646
기본 MSE: 0.9351
KFold MSE 평균: 0.9356
```

교차 검증을 통한 모델 안정성 확보

- KFold(n_splits=5)를 적용하여 5개 폴드에서 R² 및 MSE 측정
- 폴드별 R² 편차 약 0.0008 → 매우 안정적인 모델
- 기본 R²(0.9652) vs KFold 평균 R²(0.9646): 차이 0.0006 → 과적합 없음
- 회귀 문제는 연속형 값 예측으로 클래스 불균형 개념이 없어 KFold 선택

gird_search 하이퍼파라미터 설정

```
# 하이퍼파라미터 설정
param_grid = {
    'fit_intercept': [True, False], # 절편 포함 여부
    'positive': [True, False] # 계수를 양수로만 제한할지
}
```

조합 1: fit_intercept=True, positive=True

조합 2: fit_intercept=True, positive=False

조합 3: fit_intercept=False, positive=True

조합 4: fit_intercept=False, positive=False

=> 4가지 조합에 대해서 Grid_Search 자동 테스트 진행

```
GridSearch 최적 R제곱: 0.9646220778909953
기본 LinearRegression R² : 0.9652
KFold R² 평균: 0.9646
GridSearchCV 최적 R² : 0.9652
```

Grid_search란?

- 하이퍼파라미터를 자동으로 찾아주는 최적화 도구
(모든 조합을 자동으로 전부 테스트)

- fit_intercept=True → 기본 지출(절편)이 존재하는 현실 반영
- positive=True → 계수가 반드시 양수(장바구니가 클 수록, 돈을 더 많이 소비함)

- 최적화 파라미터를 찾았지만 **기본 모델 성능과 똑같은 결과**가 나옴
- 하이퍼파라미터 튜닝 효과가 크지 않았음